



16. Python: Date and Time Functions

"Working with Date and Time in Python"

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 15. Python: Mathematical Functions](#)

Contents:

[1. Common Time Functions](#)

[2. Extract Time Elements](#)

[3. Time Conversion](#)

[4. Common Date Functions](#)

[5. The `calendar` Module](#)

[Summary](#)

[References](#)

Python allows us to work not only with time, but also with dates. In real-world applications, time and date handling are extremely important because almost every system depends on them in some way: Calendars; Schedulers; Games; Banking systems; Websites; Logs and databases; Timers and reminders

Python provides built-in modules such as:

- `time` → focuses more on timestamps, runtime measurement, delays, and performance
- `datetime` → focuses more on dates, calendars, formatting, and advanced date operations

The beginning of time in Python starts from a reference point called the **Epoch**:

```
January 1, 1970 – 12:00 AM (UTC)
```

We can import the modules like this:

```
import time
from datetime import datetime, date
```

- The `time` module is generally used for: Runtime measurements; Delays; Timers; Performance testing
- Meanwhile, the `datetime` module is more suitable for: Calendar dates; Scheduling systems; Formatting dates; Date calculations

1. Common Time Functions

- `time.time()` : Returns the current time in seconds since the Epoch.

This is commonly used for measuring code runtime; building timers; simple games; stopwatch systems

```
import time

current_time = time.time()
print(current_time)
```

```
1716540000.12345
```

One fun use of `time.time()` is creating a simple math game that measures how long the player takes to finish.

```

import time

score = 0

print("=== Addition Game ===")
print("Answer 10 questions as fast as possible!")

start_time = time.time()

for i in range(1, 11):
    num1 = i * 2
    num2 = i * 3
    answer = int(input(f"Question {i}: {num1} + {num2} = "))

    if answer == num1 + num2:
        print("Correct!")
        score += 1
    else:
        print("Wrong!")

end_time = time.time()
total_time = end_time - start_time

print("\nGame Finished!")
print(f"Score: {score}/10")
print(f"Time Taken: {total_time:.2f} seconds")

```

Run this code for full output...

Game Finished!

Score: 10/10

Time Taken: 29.51 seconds

- `time.sleep(seconds)`: Pauses the program for a number of seconds. This is extremely useful in: Games; Animations; Simulations; Delayed execution systems

```
import time

print("Start")
time.sleep(2)
print("End")
```

Start
(waits 2 seconds)
End

One of the best and most common examples of `time.sleep()` is building a countdown timer.

```
import time

count = 10

while count > 0:
    print(count)
    # time.sleep(1) pauses the program for exactly 1 second
    # After waiting, the counter decreases by 1
    # The loop repeats until the countdown reaches 0
    time.sleep(1)
    count -= 1

print("Time's up!")
```

10
9
8
7
6
5
4
3
2
1
Time's up!

- `time.ctime()` : Converts seconds since Epoch into a readable time string.

```
print(time.ctime())
```

Sun May 24 01:30:00 2026

- `time.asctime()` : Converts a time tuple into a readable string.

```
current = time.localtime()  
print(time.asctime(current))
```

Sun May 24 01:43:47 2026

- `time.localtime()` : Returns local time as a `struct_time` object.

```
current = time.localtime()  
print(current)
```

`time.struct_time(tm_year=2026, tm_mon=5, tm_mday=24, tm_hour=1, tm_min=44, tm_sec=30, tm_wday=6, tm_yday=144, tm_isdst=0)`

- `time.gmtime()` : Returns UTC/GMT time instead of local time.

```
print(time.gmtime())
```

`time.struct_time(tm_year=2026, tm_mon=5, tm_mday=24, tm_hour=1, tm_min=44, tm_sec=46, tm_wday=6, tm_yday=144, tm_isdst=0)`

- `time.strftime()` : Formats time into custom strings.

```
current = time.localtime()

formatted = time.strftime("%Y-%m-%d %H:%M:%S", current)
print(formatted)
```

2026-05-24 01:46:31

Common format symbols:

Symbol	Meaning
%Y	Full year
%m	Month
%d	Day
%H	Hour
%M	Minute
%S	Second

- `time.strptime()` : Converts a string into structured time.

```
date_string = "24-05-2026"

converted = time.strptime(date_string, "%d-%m-%Y")
print(converted)
```

```
time.struct_time(tm_year=2026, tm_mon=5, tm_mday=24, tm_hour=0, tm_min=0, tm_sec=0,
tm_wday=6, tm_yday=144, tm_isdst=-1)
```

- `time.mktime()` : Converts structured time into seconds since Epoch.

```
current = time.localtime()
print(time.mktime(current))
```

- `struct_time` : A special object used to store time information.

Example:

```
current = time.localtime()

print(current.tm_year)
print(current.tm_mon)
print(current.tm_mday)
```

```
2026
5
24
```

- `time.perf_counter()` : A high-precision timer mainly used for benchmarking code performance.

```
import time

start = time.perf_counter()

for i in range(1000000):
    pass

end = time.perf_counter()

print(end - start)
```

```
0.06867532408796251
```

Another example: Fibonacci Performance Benchmark

```

import time

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

start = time.perf_counter()
result = fibonacci(35)
end = time.perf_counter()

print("Fibonacci Result:", result)
print(f"Execution Time: {end - start:.6f} seconds")

```

Fibonacci Result: 9227465

Execution Time: 1.451222 seconds

This is usually better than `time.time()` for performance testing because it provides more accurate timing.

It is especially useful for: Recursive algorithms; Sorting benchmarks; Machine Learning experiments; Performance optimization

- `time.process_time()`: Measures only CPU processing time.

```

start = time.process_time()

for i in range(1000000):
    pass

end = time.process_time()

print(end - start)

```

0.047692268999999995

Unlike `time.time()`, this does not include sleeping or waiting time.

2. Extract Time Elements

When using `struct_time`, we can extract individual time components.

```
import time

current = time.localtime()

print("Second:", current.tm_sec)    # Second
print("Minute:", current.tm_min)    # Minute
print("Hour:", current.tm_hour)     # Hour
print("Day:", current.tm_mday)      # Day
print("Month:", current.tm_mon)     # Month
print("Year:", current.tm_year)     # Year
```

Second: 33

Minute: 49

Hour: 1

Day: 24

Month: 5

Year: 2026

Common attributes:

Attribute	Meaning
<code>tm_sec</code>	Seconds
<code>tm_min</code>	Minutes
<code>tm_hour</code>	Hours
<code>tm_mday</code>	Day of month
<code>tm_mon</code>	Month
<code>tm_year</code>	Year

This is useful when: Creating clocks; Building calendars; Making countdown timers; Logging events

3. Time Conversion

Besides formatting time, Python also allows us to convert between different units of time such as: Hours; Minutes; Seconds; Milliseconds. A practical way is using the `time` module itself.

- Example: Convert Seconds into Minutes

```
import time

start = time.time()
time.sleep(120)
end = time.time()

elapsed_seconds = end - start

minutes = elapsed_seconds / 60
print("Seconds:", elapsed_seconds)
print("Minutes:", minutes)
```

Seconds: 120

Minutes: 2

This example: Starts a timer; Waits for 120 seconds; Measures the elapsed time; Converts the result into minutes.

- Example: Convert Seconds into Milliseconds

```
import time

start = time.time()
time.sleep(2)
end = time.time()

elapsed_seconds = end - start

milliseconds = elapsed_seconds * 1000
print("Milliseconds:", milliseconds)
```

Milliseconds: 2000.1108646392822

- Example: Simple Stopwatch Conversion

```
import time

start = time.time()
input("Press Enter to stop the timer...")
end = time.time()

elapsed = end - start

minutes = elapsed / 60
hours = elapsed / 3600
print(f"Seconds: {elapsed:.2f}")
print(f"Minutes: {minutes:.2f}")
print(f"Hours: {hours:.4f}")
```

Press Enter to stop the timer...

Seconds: 4.40

Minutes: 0.07

Hours: 0.0012

This is actually the foundation of: Stopwatches; Speedrun timers; Quiz systems; Game timers; Performance measurements

- Visual UTC Offset Comparison: Using `time.gmtime()` and `time.localtime()`

`gmtime()` returns UTC/GMT time, while `localtime()` returns your computer's local timezone.

```
import time

utc = time.gmtime()
local = time.localtime()

print("UTC Time:")
print(time.asctime(utc))

print("\nLocal Time:")
print(time.asctime(local))
```

UTC Time:

Sun May 24 02:02:01 2026

Local Time:

Sun May 24 02:02:01 2026

Without proper timezone handling, applications can display completely incorrect schedules and timestamps.

4. Common Date Functions

The `datetime` module provides many useful functions for working with dates and time.

- `datetime.now()` : Returns the current local date and time.

```
from datetime import datetime

current = datetime.now()
print(current)
```

2026-05-24 02:12:14.213227

- `date.today()` : Returns today's date.

```
from datetime import date

today = date.today()
print(today)
```

2026-05-24

- Extracting Date Elements: We can extract specific components from a date.

```
from datetime import datetime

current = datetime.now()
print(current.year)
print(current.month)
print(current.day)
```

2026

5

24

Common attributes:

Attribute	Meaning
<code>.year</code>	Current year
<code>.month</code>	Current month
<code>.day</code>	Current day

Attribute	Meaning
<code>.hour</code>	Current hour
<code>.minute</code>	Current minute
<code>.second</code>	Current second

- `strftime()` : Formats dates into readable strings.

```
from datetime import datetime

current = datetime.now()

formatted = current.strftime("%d/%m/%Y")

print(formatted)
```

24/05/2026

Common Format Codes

Code	Meaning
<code>%Y</code>	Full year
<code>%m</code>	Month
<code>%d</code>	Day
<code>%H</code>	Hour (24-hour format)
<code>%M</code>	Minute
<code>%S</code>	Second

- `strptime()` : Converts strings into datetime objects.

```

from datetime import datetime

date_string = "24-05-2026"

converted = datetime.strptime(date_string, "%d-%m-%Y")
print(converted)

```

2026-05-24 00:00:00

- Date Difference: We can subtract dates to calculate durations.

```

from datetime import datetime

date1 = datetime(2026, 5, 24)
date2 = datetime(2026, 6, 1)

difference = date2 - date1
print("Day Differences:", difference.days)

```

Day Differences: 8

This is very useful for: Countdown systems; Deadlines; Schedulers; Booking systems

- UTC and Regional Time Conversion: Python can also convert UTC time into other regional time zones.

Different countries use different UTC offsets:

Region	UTC Offset
Vietnam	UTC +7
Japan	UTC +9
United Kingdom	UTC +0
Germany	UTC +1
New York (USA)	UTC -5

Region	UTC Offset
California (USA)	UTC -8

```

from datetime import datetime, timedelta

utc_time = datetime.utcnow()

vietnam_time = utc_time + timedelta(hours=7)
japan_time = utc_time + timedelta(hours=9)
uk_time = utc_time + timedelta(hours=0)
german_time = utc_time + timedelta(hours=1)
america_ny_time = utc_time - timedelta(hours=5)
america_cali_time = utc_time - timedelta(hours=8)

print("UTC:", utc_time)
print("Vietnam:", vietnam_time)
print("Japan:", japan_time)
print("United Kingdom:", uk_time)
print("Germany:", german_time)
print("New York - United States:", america_ny_time)
print("California - United States:", america_cali_time)

```

```

UTC: 2026-05-24 01:53:27.214431
Vietnam: 2026-05-24 08:53:27.214431
Japan: 2026-05-24 10:53:27.214431
United Kingdom: 2026-05-24 01:53:27.214431
Germany: 2026-05-24 02:53:27.214431
New York - United States: 2026-05-23 20:53:27.214431
California - United States: 2026-05-23 17:53:27.214431

```

`datetime.utcnow()` is now replaced with `datetime.now()` in newer Python version

This is extremely important for: Websites; Multiplayer games; International applications; Global servers; Scheduling systems

5. The `calendar` Module

Another powerful built-in module in Python is the `calendar` module.

While the `time` and `datetime` modules mainly focus on timestamps, runtime, and date manipulation, the `calendar` module is specifically designed for: Monthly calendars; Yearly calendars; Weekday management; Leap year calculations; Calendar formatting

We can import it like this:

```
import calendar
```

Common Calendar Functions

- `calendar.month(year, month)` : Displays the calendar of a specific month.

```
print(calendar.month(2026, 5))
```

```
May 2026
Mo Tu We Th Fr Sa Su
    1 2 3
 4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
```

- `calendar.calendar(year)` : Displays the entire calendar of a year.

```
print(calendar.calendar(2026))
```

This is useful when creating: Schedulers; Planner applications; School systems; Booking systems

- `calendar.weekday(year, month, day)` : Returns the weekday as an integer.

Weekday Mapping:

Number	Day
0	Monday
1	Tuesday

Number	Day
2	Wednesday
3	Thursday
4	Friday
5	Saturday
6	Sunday

```
print(calendar.weekday(2026, 5, 24))
```

6

- `calendar.setfirstweekday()` : Changes the first day of the week.

```
calendar.setfirstweekday(calendar.SUNDAY)
print(calendar.month(2026, 5))
```

```
May 2026
Su Mo Tu We Th Fr Sa
      1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
```

Now the calendar starts from Sunday instead of Monday.

- `calendar.firstweekday()` : Returns the current first weekday setting.

```
print(calendar.firstweekday())
```

0

This means Sunday is currently set as the first weekday.

Leap Year Functions

Leap years are important because some years contain 366 days instead of 365. A leap year usually occurs every 4 years.

- `calendar.isleap(year)` : Checks whether a year is a leap year.

```
print(calendar.isleap(2024))  
print(calendar.isleap(2025))  
print(calendar.isleap(2026))
```

True
False
False

- `calendar.leapdays(y1, y2)` : Counts how many leap years exist between two years.

```
print(calendar.leapdays(2000, 2026))
```

7

- **Advanced Example: Display Current Month Calendar**

```
import calendar
from datetime import datetime

current = datetime.now()

print(calendar.month(current.year, current.month))
```

```
Mo Tu We Th Fr Sa Su
      1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
```

This combines both: `datetime` and `calendar` to display the current month's calendar automatically.

Summary

In this section, we explored Python's built-in modules for handling: Time; Dates; Calendars

We learned how:

- The `time` module focuses on timers, runtime measurement, delays, and performance benchmarking
- The `datetime` module focuses on dates, formatting, calculations, and scheduling
- The `calendar` module focuses on displaying and managing calendar-related information

We also explored:

- Runtime measurement using `time.time()` and `time.perf_counter()`
- Countdown systems using `time.sleep()`
- Date formatting with `strftime()` and `strptime()`
- UTC and timezone conversion
- Leap year detection

- Monthly and yearly calendar generation

Together, these modules form the foundation of time-based programming in Python and are widely used in: Websites; Games; Banking systems; Scheduling software; Scientific applications; Real-world automation systems

References

<https://docs.python.org/3/library/time.html>

<https://docs.python.org/3/library/datetime.html>

<https://docs.python.org/3/library/calendar.html>

<https://www.geeksforgeeks.org/python/python-time-module/>

<https://www.geeksforgeeks.org/python/python-datetime-module/>

<https://www.geeksforgeeks.org/python/python-calendar-module/>

Next Section:

[!\[\]\(5db460c3746afb1ce6e75bddb304caae_img.jpg\) 17. Python: Random Numbers](#)